PHP Full Stack Assignment

Module 1 –Overview of IT Industry

Q1.: Write a simple "Hello World" program in two different programming languages of your choice. Compare the structure and syntax.

ANS: C lanuguages

```
#include <stdio.h>
        int main()
        {
    printf("Hello, World!\n");
     return 0;
  }
```

2:Python

```
     print("Hello, World!")
```

C LANGUAGES STRUCTURE AND SYNTAX

#include<stdio.h> include the standard input/output library

Int main() The program entery point

Print f(): Used to point output to the screen

Return 0; indicates successful execution

Python languages structure and syntax

Just one line to print text

No need for main function or class

Q2. Research and create a diagram of how data is transmitted from a client to a server over the internet.

Ans: [Client Device]

|

| 1. Request (URL)

v

[DNS Server] <-- Resolves domain name to IP address

|

| 2. IP Address returned

v

[Internet Backbone]

|

| 3. Data sent via TCP/IP

v

[Web Server]

|

| 4. Sends HTTP/HTTPS Response

v

[Client Device]

|

| 5. Browser renders page

v

[User sees content]

Q3. Simulate HTTP and FTP requests using command line tools (e.g., curl).

Ans: Simulating HTTP Requests with **curl**:

:GET Request (basic).

To retrieve the content of a web page, use **curl** followed by the URL

```
curl https://www.example.com
```

:GET Request (saving to file).

To save the retrieved content to a file, use the **-o** option

```
curl -o homepage.html https://www.example.com
```

: POST Request.

To send data using a POST request, use the **-X POST or --request POST** option along with the **-d or --data** option for the data

```
curl -X POST -d "username=user&password=pass" https://api.example.com/login
```

: Including Headers.

To add custom HTTP headers, use the **-H** option

```
curl ftp://ftp.example.com/path/to/file.txt
```

;Simulating FTP Requests with **curl:**

Downloading a File. with Authentication.

To download a file from an FTP server, specify the FTP URL:

```
curl ftp://ftp.example.com/path/to/file.txt
```

;Uploading a File.

To upload a file to an FTP server, use **the -T** option followed by the local file path:

```
curl -u username:password -T local_file.txt ftp://ftp.example.com/remote_directory/
```

;Listing Directory Contents.

To list the contents of an FTP directory, specify the directory path

```
curl ftp://ftp.example.com/remote_directory/
```

Q.4 Design a simple HTTP client-server communication in any language.

Ans: A Server (using Python's built-in http.server)

A Client (using the requests library)

HTTP Server (server.py)

```
from http.server import Base HTTP Request Handler, HTTP Server

class Simple Handler(Base HTTP Request Handler):

    def do_GET(self):

        # Send response status code

self.send_response(200)
```

```
        # Send headers
self.send_header('Content-type', 'text/html')
self.end_headers()

        # Send the body of the response
self.wfile.write(b"Hello from the Server!")

# Server settings
host = "localhost"
port = 8080

server = HTTP Server((host, port), Simple Handler)
print(f"Serving on http://{host}:{port}")
server.serve_forever()
```

HTTP Client (client.py)

```
import requests
# Make a GET request to the server
response = requests.get("http://localhost:8080")
```

Print the response text

print("Server says:", response.text)

How to Run

Save the server code as server.py and run:

python server.py

(It will start listening at http://localhost:8080.)

In a separate terminal, save the **client code** as client.py and run:

python client.py

(It will send a GET request to the server and print the response.)

What Happens

Client → Sends GET / request to localhost:8080.

Server → Receives request, sends back "Hello from the Server!".

Client → Prints the server's response

Q5. : **Research different types of internet connections (e.g., broadband, fiber, satellite)and list their pros and cons.**

Ans. Different types of internet connections offer varying speeds and reliability. Broadband, including fiber and cable, generally provides the fastest speeds, while satellite and DSL may be better suited for rural areas or where fiber isn't available. Satellite internet excels in availability but can be affected by weather and latency

1. Fiber Optic:

- **Pros:** Fastest speeds (up to Gigabit+), low latency, very reliable, symmetrical speeds (upload and download are the same).
- **Cons:** Most expensive, availability can be limited, installation can be complex.
- 2. Cable:
- **Pros:** High speeds, good availability, reliable for most users.
- **Cons:** Speeds can vary based on network usage, potentially slower than fiber.
- 3. DSL:
- **Pros:** Affordable, good for light users, widely available.
- **Cons:** Slower speeds compared to fiber and cable, susceptible to distance from the provider's equipment.
- 4. Satellite:
- **Pros:** Available virtually anywhere, good for rural areas.
- **Cons:** Most expensive, slowest speeds, high latency, affected by weather conditions.
- 5. Fixed Wireless:
- **Pros:** Decent speeds, can be a good option for rural areas, more affordable than satellite.
- **Cons:** Requires line of sight to the provider's tower, speeds can fluctuate.
- 6. 5G Home Internet:
- **Pros:** Relatively fast speeds, good for urban and suburban areas.
- **Cons:** Speeds can fluctuate, may not be as reliable as fiber or cable.

Q6. : Identify and explain three common application security vulnerabilities. Suggestpossible solutions.

Ans: Three common application security vulnerabilities are SQL injection, Cross-Site Scripting (XSS), and insecure direct object references (IDOR). SQL injection allows attackers to execute malicious SQL code by injecting it into input fields, potentially gaining unauthorized access to data or even the entire database. XSS enables attackers to inject malicious scripts into websites

1. SQL Injection (SQLi):

- **Vulnerability:**

  Attackers exploit vulnerabilities in how applications handle user input to manipulate database queries. They can insert malicious SQL code into input fields, such as login forms or search bars, which is then executed by the application's database.

- **Impact:**

  Successful SQL injection attacks can lead to data breaches, data modification, denial of service, or even complete system compromise.

- **Solution:**
  Use prepared statements (parameterized queries) or ORM frameworks to sanitize user input and prevent it from being interpreted as SQL code. Also, validate user input to ensure it conforms to expected formats and lengths.

2. Cross-Site Scripting (XSS):

- **Vulnerability:**

  Attackers inject malicious scripts (typically JavaScript) into web pages viewed by other users. These scripts can be executed in the user's browser, potentially stealing session cookies, redirecting users to malicious sites, or modifying the page content.

- **Impact:**

  XSS attacks can lead to session hijacking, data theft, defacement of websites, and redirection to phishing sites.
- **Solution:**
  Escape user-provided data before rendering it on the page. Implement strict input validation to filter out potentially malicious scripts. Use Content Security Policy (CSP) to restrict the sources from which scripts can be loaded.

  3. Insecure Direct Object References (IDOR):
- **Vulnerability:**

  IDOR vulnerabilities arise when an application exposes direct references to internal objects (e.g., database IDs, file paths) in URLs or form fields. An attacker can manipulate these references to access or modify resources they are not authorized to access.
- **Impact:**

  IDOR attacks can lead to unauthorized access to sensitive data, modification of data, or even the ability to delete or modify other users' accounts.
- **Solution:**
  Implement strong access control mechanisms to ensure that users can only access resources they are authorized to. Use randomly generated IDs or tokens instead of direct object references.

Q7. Identify and classify 5 applications you use daily as either system software orapplication software.

Ans. System Software:

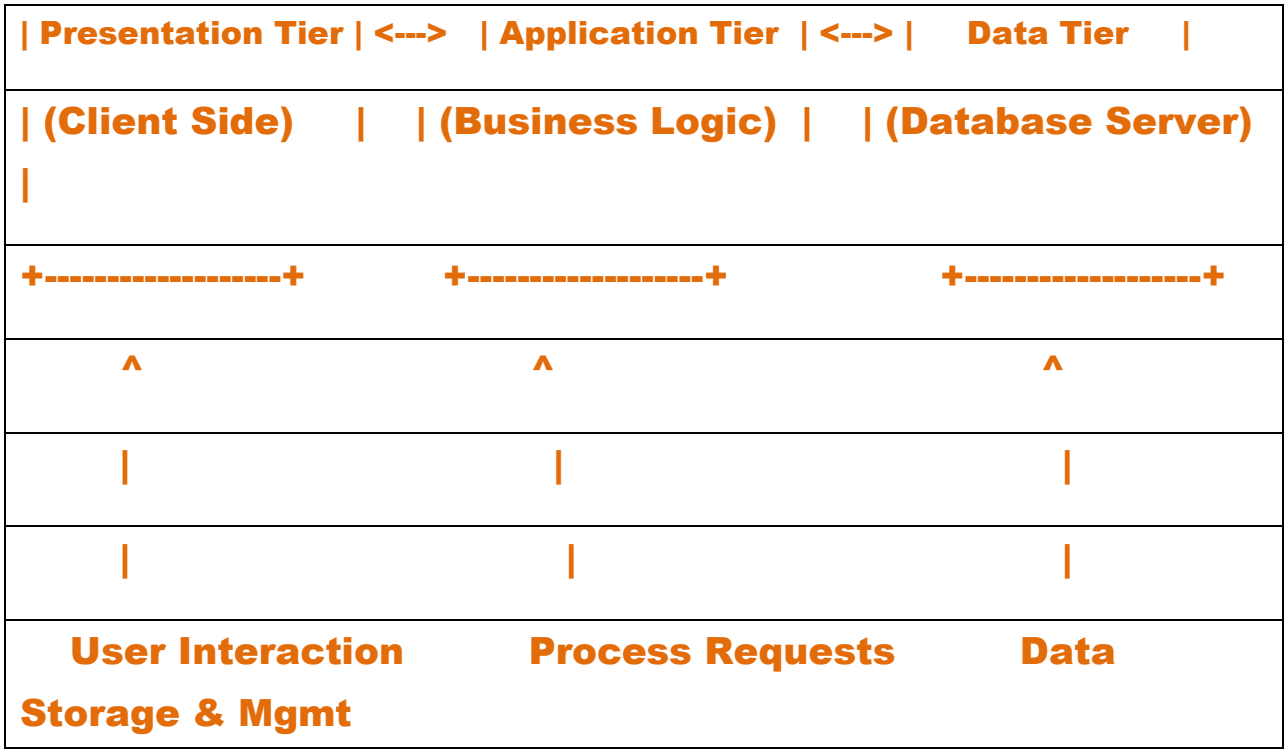
1. **Operating System (Windows):** This manages the computer's resources and provides a platform for all other software to run. It's essential for the computer to function.

Application Software:

1. **Web Browser (Google Chrome):** This allows me to access and interact with websites and online content.
2. **Email Client (Gmail):** This application is used to send, receive, and manage emails.
3. **Word Processor (Microsoft Word):** This allows me to create and edit documents.
4. **Instant Messaging App (Slack):** This facilitates communication and collaboration with others through text-based messages.

Q8. Design a basic three-tier software architecture diagram for a web application

Ans.



1. Presentation Tier (Client Side):

This tier is the user interface of the application. It is responsible for displaying information to the user and receiving user input. This typically includes:

- **Web Browser:** Renders HTML, CSS, and JavaScript.
- **User Interface Frameworks/Libraries:** (e.g., React, Angular, Vue.js) for building interactive UIs.]
-

2. Application Tier (Business Logic):

Also known as the "middle tier" or "logic tier," this tier handles the core business logic and processing. It acts as an intermediary between the Presentation Tier and the Data Tier. Components in this tier include:

- **Web Server:** (e.g., Apache, Nginx) for serving static content and routing requests.
- **Application Server:** (e.g., Node.js, Spring Boot, Django, Ruby on Rails) that executes the business logic, handles user authentication, authorization, and interacts with the Data Tier.
- **APIs/Services:** Define how the Presentation Tier communicates with the Application Tier.

3. Data Tier (Database Server):

This tier is responsible for storing, managing, and retrieving data for the application. It ensures data persistence and integrity. Key components are:

- **Database Management System (DBMS):** (e.g., MySQL, PostgreSQL, MongoDB, SQL Server) for managing the database.
- **Database:** The actual storage of application data.
- **Data Access Layer (DAL):** (often part of the Application Tier but interacts directly with the Data Tier) responsible for abstracting database interactions and providing a consistent interface for the Application Tier.

Q8. Create a case study on the functionality of the presentation, business logic, and dataaccess layers of a given software system.

Ans. 1. Presentation Layer:

- **Functionality:**

  This layer is responsible for the user interface and all aspects of user interaction. In an e-commerce application, this includes displaying product catalogs, handling user input for search queries, managing the shopping cart interface, and facilitating the checkout process. It also handles user authentication and registration forms.

- **Interaction:**
  The presentation layer communicates with the business logic layer by sending user requests (e.g., "add to cart," "place order," "search product") and receiving data to display (e.g., product details, order

confirmation). It often utilizes web technologies like HTML, CSS, and JavaScript, or mobile UI frameworks.

2. Business Logic Layer (BLL):

- **Functionality:**

  The BLL contains the core business rules and processes of the e-commerce application. This includes validating user inputs, calculating product prices and discounts, managing inventory levels, processing orders, handling payment gateway integrations, and applying business rules for promotions or shipping. It acts as an intermediary between the presentation and data access layers.

- **Interaction:**
  The BLL receives requests from the presentation layer, processes them according to defined business rules, and then interacts with the data access layer to retrieve or store necessary data. It returns processed information or status updates back to the presentation layer. For example, when an order is placed, the BLL verifies stock, calculates the total, and initiates the payment process.

3. Data Access Layer (DAL):

- **Functionality:**

  The DAL is responsible for abstracting the underlying data storage mechanism and providing a consistent interface for the BLL to interact with the database. It handles all data storage and retrieval operations, including querying product information, storing customer details, updating inventory, and recording transaction history. It manages database connections, executes SQL queries, and maps database results to application-specific objects.

- **Interaction:**
  The DAL receives requests from the BLL to perform data operations (e.g., "get product by ID," "save order," "update inventory"). It then interacts directly with the database system (e.g., SQL Server, MySQL, MongoDB) and returns the requested data or confirmation of successful operations to the BLL. This layer ensures data integrity and often includes error handling for database interactions.

Benefits of Layered Architecture:

This layered approach in the e-commerce application offers several advantages:

- **Separation of Concerns:**

  Each layer has a distinct responsibility, making the system easier to understand, develop, and maintain.

- **Modularity and Reusability:**

  Layers can be developed and tested independently, and components within a layer can be reused across different parts of the application.

- **Flexibility and Scalability:**

  Changes in one layer (e.g., updating the UI or changing the database) have minimal impact on other layers. This also facilitates scaling individual layers based on demand.

- **Testability:**
  The clear separation allows for easier unit and integration testing of each layer's functionality.

Q9. Explore different types of software environments (development, testing, production).Set up a basic environment in a virtual machine.

Ans. 1. Development Environment:

This is where developers write, debug, and test code locally. It's highly personalized to individual developer needs and often includes Integrated Development Environments (IDEs), version control systems, and local databases. Changes are typically isolated to prevent interference with other developers' work.

2. Testing Environment:

Dedicated to quality assurance (QA), this environment is used to rigorously test the application's functionality, performance, and security. It aims to replicate the production environment as closely as possible to identify bugs and ensure stability before release. Multiple

testing environments may exist for different types of testing (e.g., integration testing, user acceptance testing).

3. Production Environment:

This is the live environment where the application is deployed and accessible to end-users. It is highly optimized for performance, security, and reliability, with strict access controls and monitoring in place. Changes are deployed only after thorough testing and approval.

Setting Up a Basic Environment in a Virtual Machine:

Virtual machines (VMs) offer an isolated and reproducible way to set up these environments.

Steps:

- **Install a Hypervisor:**

  Install virtualization software like Oracle VirtualBox or VMware Workstation on your host machine.
- **Create a New Virtual Machine:**
- Launch your hypervisor and create a new VM.
- Allocate resources (RAM, CPU cores, disk space) based on your needs.
- Select an operating system (e.g., Ubuntu Server, CentOS) and mount its ISO image for installation.

Install the Operating System:

Follow the on-screen prompts within the VM to install the chosen operating system.

Install Necessary Software:

- **Development:** Install your preferred IDE (e.g., VS Code, Eclipse), programming language runtime (e.g., Node.js, Python), version control client (e.g., Git), and any required frameworks or libraries.
- **Testing:** Install testing frameworks (e.g., Selenium, JUnit), performance testing tools (e.g., JMeter), and any necessary data generation tools.

Configure Networking:

Configure network settings within the VM to allow access to external resources or other VMs as needed (e.g., bridged adapter for direct network access, NAT for shared internet).

Snapshot the VM:

Create a snapshot of the VM after initial setup to easily revert to a clean state if needed.

Q10. Write and upload your firstsource code file to Github

- **Ans. Create a local project directory and file:**

  Create a new folder on your computer for your project. Inside this folder, create your source code file
  (e.g **hello.py, index.html, main.cpp**) and add your code. Initialize a Git repository.

  Open your terminal or command prompt, navigate to your project directory, and initialize a Git repository:

  Code: git init

  Stage your file.

  Add your source code file to the Git staging area

  Code: git add <your_file_name>

  # Or to add all files in the directory:

  # git add .

  Commit your changes.

  Commit the staged file with a descriptive message

  Code: git commit -m "Initial commit of my first source code file"

Q11. Create a Github repository and document how to commit and push code changes.

Ans. Open your terminal or command prompt, navigate to your project directory, and run

Code: git init

2 add files to the staging area.

Add the files you want to include in your commit to the staging area. To add all changes in the current directory

Code: git add .

3 To add specific files

Code: git add <filename>

4 Commit the Changes.

Commit the staged changes to your local repository with a descriptive message:

Code: git commit -m "Your descriptive commit message here"

Q12. : **Create a student account on Github and collaborate on a small project with aclassmate.**

Ans. Create a GitHub Student Account

1. **Go to GitHub Education.**
2. **Click "Get Student Benefits".**
3. **Sign in with your existing GitHub account or create a new one.**

4. Fill out the form:

- School-issued email address (if you have one)

OR upload a photo of your student ID.

- Proof of enrollment (student ID card, transcript, or acceptance letter).

5. Wait for approval (usually within a few days).

6. Once approved, you'll get free access to:

- Private repositories
- GitHub Pro features
- Developer tools (e.g., free hosting, CI/CD tools).

2. Create a Project Repository

1. Log in to GitHub → Click the + → New Repository.
2. Give it a name, e.g., class-project.
3. Choose Public (or Private if you want only teammates to see it).
4. Initialize with:

- README file (project description)
- .gitignore (optional, for ignoring certain files)

5. Click Create Repository.

3. Add Your Classmate as a Collaborator

1. Go to your repository's Settings → Collaborators.
2. Click Add People.
3. Enter your classmate's GitHub username.
4. Click Add Collaborator (they will receive an invite).

4. Cloning the Repository (Both Students)

On each computer:

git clone https://github.com/YOUR-USERNAME/class-project.git

cd class-project

5. Working Together

- **Student A edits index.html and commits:**
- **git add index.html**
- **git commit -m "Added homepage content"**
- **git push**
- **Student B pulls latest changes before editing:**
- **git pull**
- **Repeat the process for every change.**

6. Avoid Conflicts

- **Always run:**
- **git pull**

before starting new changes.

- **Commit often with clear messages.**
- **Use branches for separate features, then merge**

7. Example Project Idea

- **A simple HTML + CSS personal profile website.**
- **Student A: Designs homepage.**
- **Student B: Creates “About” page.**

- **Merge both into the main branch.**

Q13 **Create a list of software you use regularly and classify them into the followingcategories: system, application, and utility software.**

System Software:

- **Operating System:**

  Windows, macOS, Linux (e.g., Ubuntu, Fedora), Android, iOS. These provide the fundamental platform for all other software to run, managing hardware and software resources.
- **Device Drivers:**
  Software components that enable the operating system to communicate with and control specific hardware devices (e.g., graphics card drivers, printer drivers).
  Application Software:
- **Web Browsers:**

  Google Chrome, Mozilla Firefox, Microsoft Edge, Safari. Used for accessing and navigating the internet.
- **Office Suites:**

  Microsoft Office (Word, Excel, PowerPoint, Outlook), Google Workspace (Docs, Sheets, Slides, Gmail). Used for productivity tasks like document creation, data analysis, presentations, and email.
- **Communication Software:**

  Zoom, Microsoft Teams, Slack, Discord. Used for video conferencing, instant messaging, and team collaboration.
- **Multimedia Software:**

  VLC Media Player, Spotify, Adobe Photoshop, Adobe Premiere Pro. Used for consuming or creating various forms of media, including audio, video, and images.
- **Development Environments:**

Visual Studio Code, PyCharm, Eclipse. Used by programmers for writing, testing, and debugging software.
Utility Software:

- **Antivirus/Anti-malware Software:**

  Windows Defender, Avast, Malwarebytes. Protects the system from malicious software.

- **Disk Management Tools:**

  Disk Cleanup, Disk Defragmenter (on older systems), file compression utilities (e.g., WinRAR, 7-Zip). Used for optimizing disk space and system performance.

- **Backup and Recovery Software:**
  Windows Backup and Restore, Time Machine. Used for creating backups of data and restoring them in case of data loss.

Q14 Follow a GIT tutorial to practice cloning, branching, and merging repositories.

Ans. 1. Create a New Repository

You can either do this on GitHub or locally.

On GitHub:

1. Go to github.com.

2. Click New repository.

3. Name it (e.g., git-branching-tutorial), keep it public, and check Initialize with a README.

4. Copy the repository's URL.

Locally:

git clone https://github.com/YourUsername/git-branching-tutorial.git
cd git-branching-tutorial

2. Check Your Current Branch

git branch

By default, you'll be on main (or master).

3. Create a New Branch

Let's create a branch called feature-1:

git checkout -b feature-1

This:

Creates a new branch (feature-1)

Switches to it

4. Make Changes

Edit README.md or create a new file:

echo "This is feature 1 work." >> feature1.txt

git add feature1.txt
git commit -m "Add feature 1 file"

5. Switch Back to Main

git checkout main

6. Merge the Feature Branch

git merge feature-1

If there are no conflicts, Git will merge automatically.

7. Push Changes to GitHub

git push origin main

If you want to also push the branch:

git push origin feature-1

8. Practice Conflict Resolution

1. On main branch, edit feature1.txt and commit

2. Switch to feature-1, make a different edit in the same part of the file, and commit.

3. Merge feature-1 into main:

git checkout main

git merge feature-1

4. Git will report a merge conflict. Open the file, resolve it manually, then:

git add feature1.txt
git commit

9. Delete a Branch (optional)

After merging, you can delete feature-1:

git branch -d feature-1

Q15 : Write a report on the various types of application software and how they improveproductivity.
Ans. Types of Application Software:

- **Productivity Software:**

  This category includes tools like word processors (Microsoft Word, Google Docs), spreadsheets (Excel, Google Sheets), and presentation software (PowerPoint, Google Slides). These applications enable efficient document creation, data analysis, and presentation development, significantly boosting individual and team productivity.
- **Communication and Collaboration Software:**

  Platforms like Slack, Microsoft Teams, Zoom, and Google Meet facilitate real-time communication and collaboration, enabling teams to connect, share information, and work together seamlessly regardless of location.
- **Project Management Software:**

Tools like Asana, Trello, and Jira help teams plan, track, and manage projects, ensuring tasks are completed on time and within budget.

- **Customer Relationship Management (CRM) Software:**

  CRM systems, such as Salesforce and HubSpot, help businesses manage customer interactions, track sales pipelines, and improve customer service, ultimately driving business growth and efficiency.

- **Content Management Systems (CMS):**

  Platforms like WordPress and Drupal enable users to create, manage, and publish website content, streamlining content creation and website management processes.

- **Accounting Software:**

  Applications like QuickBooks and Xero automate financial tasks, manage invoices, track expenses, and generate financial reports, improving the accuracy and efficiency of financial operations.

- **Enterprise Resource Planning (ERP) Software:**
  ERP systems integrate various business processes, such as finance, HR, and supply chain management, providing a holistic view of the organization and streamlining operations.
  II. How Application Software Improves Productivity:

- **Automation of Tasks:**

  Application software automates repetitive tasks, freeing up human capital for more strategic and creative work. For example, automating invoice generation or data entry tasks reduces manual effort and minimizes errors.

- **Streamlined Workflows:**

  By centralizing information and automating processes, application software streamlines workflows, making it easier for individuals and teams to work efficiently.

- **Enhanced Collaboration:**

  Communication and collaboration tools facilitate seamless teamwork, regardless of location. Features like real-time document editing, video conferencing, and instant messaging enable teams to work

together effectively, accelerating project completion and decision-making.

- **Improved Data Management and Analysis:**

  Application software provides tools for data storage, organization, and analysis, enabling businesses to make informed decisions based on accurate and up-to-date information.

- **Increased Efficiency and Accuracy:**

  Automation, streamlined workflows, and improved data management contribute to increased efficiency and accuracy in various business processes, leading to cost savings and improved performance.

- **Reduced Errors and Rework:**

  Automation and streamlined processes minimize the risk of human error, reducing the need for rework and improving the overall quality of work.

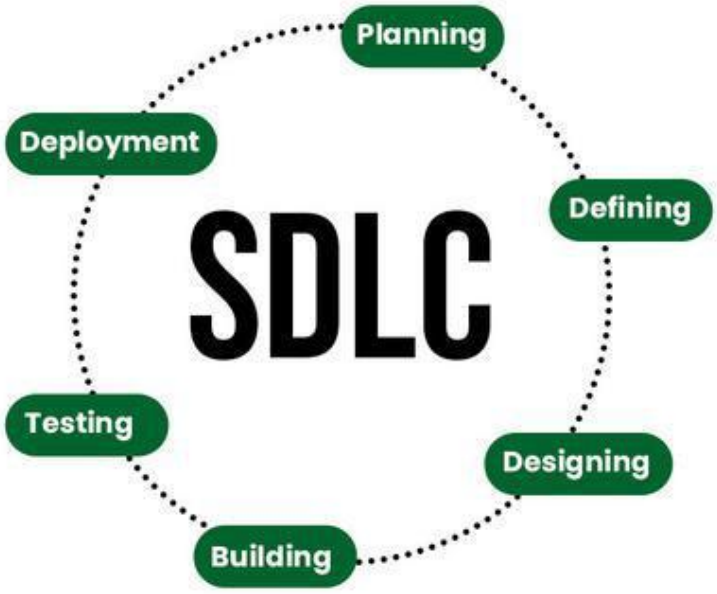
- **Better Time Management:**

  Productivity tools help individuals and teams prioritize tasks, track time spent on different activities, and manage their schedules effectively, leading to better time management and increased output.

- **Scalability and Flexibility:**
  Application software, especially cloud-based solutions, provides scalability and flexibility, allowing businesses to adapt to changing needs and grow their operations efficiently.

Q16 Create a flowchart representing the Software Development Life Cycle (SDLC)

Ans.



Q17. **Write a requirement specification for a simple library management system.**

ANS. Introduction

- **Purpose:**

  To outline the requirements for a system that automates the core functions of a small library, including book management, member management, and circulation.

- **Scope:**
  The system will manage book inventory, member details, book borrowing/returning, and provide basic search functionality.

2. Functional Requirements

- **Book Management:**
- Ability to add new books with details (Title, Author, ISBN, Publication Year, Genre, Number of Copies).
- Ability to update existing book details.
- Ability to remove books from the system.

- Ability to view the list of all books.
- **Member Management:**
- Ability to add new library members with details (Name, Member ID, Contact Information).
- Ability to update existing member details.
- Ability to remove members.
- Ability to view the list of all members.
- **Circulation Management:**
- Ability to issue a book to a member, recording the issue date and due date.
- Ability to return a book, marking it as available.
- Ability to track borrowed books per member.
- Ability to identify overdue books.
- **Search Functionality:**
- Ability to search for books by Title, Author, or ISBN.
- Ability to search for members by Name or Member ID.
- 3. Non-Functional Requirements
- **Usability:** The system shall have an intuitive and easy-to-navigate user interface for librarians.
- **Performance:** The system shall provide quick response times for searches and transactions.
- **Security:** Access to manage books and members shall be restricted to authorized librarian users through a login mechanism.
- **Reliability:** The system shall ensure data integrity and minimize data loss.
- **Maintainability:** The system shall be designed for ease of future modifications and updates.
- 4. User Interface Requirements

- A clear and consistent graphical user interface (GUI) for all functionalities.
- Forms for data entry and display of information.
- Confirmation messages for critical actions (e.g., deleting a book).
-

5. Data Requirements

- Database to store book information (Title, Author, ISBN, Publication Year, Genre, Available Copies, Total Copies).
- Database to store member information (Name, Member ID, Contact Information).
- Database to store borrowing records (Member ID, Book ISBN, Issue Date, Due Date, Return Date).

Q18. **Perform a functional analysis for an online shopping system.**
ANSCore Functional Areas:

1. **1. User Management:**

- **Registration and Login:** Users should be able to create accounts, log in securely, and manage their profiles. This includes options for guest checkout as well.
- **User Roles:** The system should support different user roles (e.g., customer, administrator) with varying access permissions.
-

2. **2. Product Catalog and Search:**

- **Product Listing:** Display products with detailed descriptions, images, prices, and availability.
- **Search and Filtering:** Allow users to search for products using keywords and apply filters (e.g., price range, category).
-

3. **3. Shopping Cart and Checkout:**

- **Add to Cart:** Enable users to add desired products to a shopping cart.

- **Cart Management:** Allow users to view, edit (change quantities, remove items), and save items in the cart.
- **Checkout Process:** Facilitate a secure and user-friendly checkout process, including address input, shipping options, and payment methods.

4. **4. Order Management:**

- **Order Placement:** Allow users to finalize their purchases and place orders.
- **Order Tracking:** Provide customers with the ability to track the status of their orders.
- **Order History:** Enable users to view their past orders.
- **Admin Order Management:** Allow administrators to manage orders, update statuses, and handle customer inquiries.

5. **5. Payment Processing:**

- **Secure Transactions:** Implement secure payment gateways to process transactions safely.
- **Payment Methods:** Support various payment options (e.g., credit card, PayPal).

6. **6. Administrative Functions:**

- **Product Management:** Allow administrators to add, edit, and remove products from the catalog.
- **User Management:** Enable administrators to manage user accounts and roles.
- **Order Management:** Provide tools for administrators to manage all aspects of orders.
- **Reporting and Analytics:** Offer reporting and analytics features to track sales, customer behavior, and other key metrics.

7. **7. Security:**

- **Secure Data Storage:** Protect sensitive user data, including personal and payment information, with appropriate security measures.

- **Authentication and Authorization:** Implement secure authentication and authorization mechanisms to control access to the system and its resources.

8. **8. Non-Functional Requirements:**

- **Performance:** The system should be responsive and efficient, with fast loading times and quick response to user actions.
- **Scalability:** The system should be able to handle a growing number of users and transactions.
- **Availability:** The system should be available to users when they need it.
- **Accessibility:** The system should be accessible to users with disabilities.
- **Maintainability:** The system should be designed for easy maintenance and updates.

Q19 : **Design a basic system architecture for a food delivery app**.
ANS. Client Applications:

- **Customer App:**

  Mobile (iOS/Android) or web application for users to browse restaurants, view menus, place orders, make payments, and track deliveries.

- **Restaurant Partner App/Portal:**

  Application or web interface for restaurants to manage menus, accept/reject orders, update order status, and view sales data.

- **Delivery Partner App:**
  Mobile application for delivery personnel to receive delivery assignments, navigate to restaurants and customers, and update delivery status.
  2. Backend Services (Microservices Architecture Recommended):

- **User Management Service:**

  Handles user registration, authentication, authorization (customer, restaurant, delivery partner roles).

- **Restaurant Service:**

  Manages restaurant information (menus, locations, operating hours, ratings).
- **Order Management Service:**

  Processes order placement, manages order states (pending, accepted, prepared, in transit, delivered), and handles order cancellations/modifications.
- **Payment Service:**

  Integrates with payment gateways to process transactions securely.
- **Delivery Service:**

  Assigns delivery partners to orders, manages delivery routes, and provides real-time tracking.
- **Notification Service:**

  Sends push notifications, SMS, or emails for order updates, delivery status, and promotions.
- **Search and Discovery Service:**
  Enables users to search for restaurants and menu items, often using technologies like Elasticsearch for fast and relevant results.
  3. Data Storage:
- **Relational Database (e.g., PostgreSQL, MySQL):**

  Stores structured data like user profiles, order details, restaurant information, and menu items.
- **NoSQL Database (e.g., MongoDB, Cassandra):**

  Can be used for flexible data storage, such as user preferences, real-time location data for delivery partners, or analytics.
- **Caching Layer (e.g., Redis):**
  Improves performance by storing frequently accessed data, like popular menu items or restaurant details, in memory.
  4. Infrastructure:
- **API Gateway:**

Acts as a single entry point for all client requests, handling authentication, routing, and rate limiting.

- **Load Balancers:**

  Distribute incoming traffic across multiple instances of backend services to ensure scalability and high availability.

- **Message Queue (e.g., Kafka, RabbitMQ):**

  Facilitates asynchronous communication between microservices, ensuring reliable message delivery and decoupling services.

- **Cloud Platform (e.g., AWS, Google Cloud, Azure):**

  Provides scalable infrastructure, managed services for databases, messaging, and computing resources.

- **Monitoring and Logging:**
  Tools to track system performance, identify issues, and ensure operational stability.
  5. Third-Party Integrations:

- **Payment Gateways:** For processing online payments.
- **Mapping and Geolocation Services:** For displaying restaurant and delivery partner locations, calculating routes, and providing estimated delivery times.
- **SMS/Email Providers:** For sending notifications.

Q20 Develop test cases for a simple calculator program.

Ans. Basic Arithmetic Operations:

- **Addition:**
- Positive integers: `2 + 3 = 5`
- Negative integers: `-5 + (-2) = -7`
- Mixed integers: `10 + (-4) = 6`
- Zero with a number: `0 + 7 = 7`, `5 + 0 = 5`
- Decimal numbers: `2.5 + 3.7 = 6.2`

- **Subtraction:**
- Positive integers: `8 - 3 = 5`
- Negative integers: `-5 - (-2) = -3`
- Mixed integers: `10 - (-4) = 14`
- Zero with a number: `0 - 7 = -7`, `5 - 0 = 5`
- Decimal numbers: `7.5 - 2.3 = 5.2`
-
- **Multiplication:**
- Positive integers: `4 * 6 = 24`
- Negative integers: `-3 * (-5) = 15`
- Mixed integers: `7 * (-2) = -14`
- Zero with a number: `0 * 8 = 0`, `5 * 0 = 0`
- Decimal numbers: `1.5 * 2.0 = 3.0`
-
- **Division:**
- Positive integers: `10 / 2 = 5`
- Negative integers: `-12 / (-3) = 4`
- Mixed integers: `15 / (-3) = -5`
- Zero divided by a non-zero number: `0 / 5 = 0`
- Decimal numbers: `9.9 / 3.3 = 3.0`
-
- **Edge Case: Division by Zero:** `5 / 0` (should result in an error message or "Infinity")
  2. Input Handling and Display:
- **Single Digit Entry:** `7`, `0`
- **Multiple Digit Entry:** `123`, `4567`
- **Decimal Point Entry:** `1.23`, `0.5`
- **Clear Button:** Pressing "C" should reset the display to `0` and clear any pending operations.

- **Backspace/Delete:** Deleting the last entered digit.
- **Operator Overriding:** Pressing `2 + + 3` should result in `2 + 3 = 5` (the second `+` overrides the first).

Q21. Document a real-world case where a software application required criticalmaintenance.

Ans. The Problem:

- The patient management system, crucial for tracking patient information, medication orders, and medical history, suffered a database error that caused data corruption.
- This resulted in inaccurate patient records, potentially leading to incorrect medication administration, misdiagnosis, and other serious medical errors.
- The issue was discovered when clinicians noticed inconsistencies in patient information and reported the problem.
  The Maintenance Process:
- **Immediate Response:**

  The hospital IT team, along with the software vendor, initiated an emergency response to identify the root cause of the data corruption.
- **Data Recovery:**

  The team focused on recovering the corrupted data from backups and employing data repair techniques.
- **System Downtime:**

  Due to the critical nature of the issue, the system had to be taken offline for a significant period to ensure the data was fully restored and verified.
- **Code Inspection:**

  The software vendor analyzed the code to identify the bug that caused the data corruption and implemented a fix.
- **Testing and Validation:**

The restored data and the code fix were rigorously tested to ensure data accuracy and system stability before bringing the system back online.

- **Preventive Measures:**

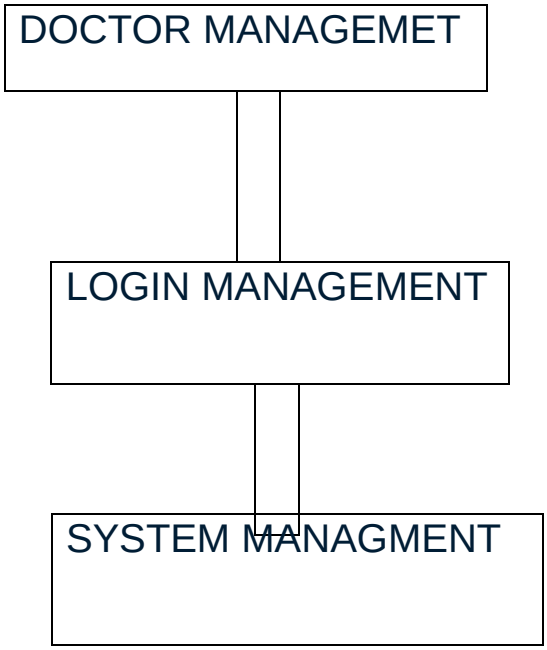
  The vendor also implemented additional data validation checks and monitoring mechanisms to prevent similar issues from occurring in the future.

- **Communication:**
  The hospital kept the medical staff informed about the progress of the maintenance and the expected time for system restoration.
  Impact:

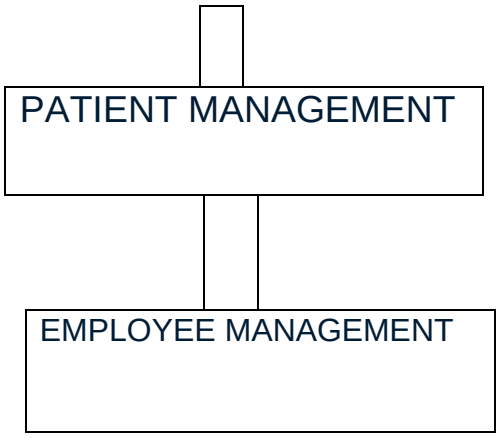
- The incident highlighted the importance of robust data backup and recovery procedures in healthcare systems.
- It emphasized the need for regular software maintenance and updates to prevent critical errors.

Q22. **Create a DFD for a hospital management system.**

Ans. HOSPITAL MANAGEMET SYSTEM



1



Q23. Draw a flowchart representing the logic of a basic online registration system